

# dp 优化 2：单调队列，斜率优化

董国梁



### 1. 本质

- 单调队列是双端队列（deque）的变种，队列内元素按**严格单调递增或递减**排列，分为单调递增队列（队头最小）和单调递减队列（队头最大）
- 通过动态维护单调性，支持在  $O(1)$  时间内获取当前队列的最值（队头元素）

### 2. 核心操作

- **入队**：新元素加入时，从队尾移除所有破坏单调性的旧元素（例：递减队列中移除比新元素小的队尾元素），再入队
- **出队**：检查队头元素是否超出窗口范围（索引  $< i-k+1$ ），若超出则弹出
- **查询极值**：队头元素即为当前窗口最值

- 给你一个长度为 $n$ 的数列，从左至右输出每个长度为 $m$ 的数列段内的最大数。
- 数列段即 $[1, m]$ ,  $[2, m+1]$ ,  $\dots$ ,  $[n-m+1, n]$ , 共 $n-m+1$ 个。

• 样例输入：

7 3  
1 5 3 2 2 4 1

• 样例输出：

5 5 3 4 4

数据范围：  $1 < n < 200000$ ,  $1 < a < 10^9$

- 我们可以用一个单调递减的队列来解决这个问题，队首元素最大。 数列：1 5 3 2 2 4 1

i=1  
动态区间[-1, 1]

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 数列   | 1 | 5 | 3 | 2 | 2 | 4 | 1 |
| 数组下标 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 动态区间 |   |   |   |   |   |   |   |

单调队列：1

i=2  
动态区间[0, 2]

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 数列   | 1 | 5 | 3 | 2 | 2 | 4 | 1 |
| 数组下标 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 动态区间 |   |   |   |   |   |   |   |

单调队列：5

- 我们可以用一个单调递减的队列来解决这个问题，队首元素最大。 数列：1 5 3 2 2 4 1

i=3  
动态区间[1, 3]

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 数列   | 1 | 5 | 3 | 2 | 2 | 4 | 1 |
| 数组下标 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

动态区间

单调队列：5 3  
最后答案：5

i=4  
动态区间[2, 4]

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 数列   | 1 | 5 | 3 | 2 | 2 | 4 | 1 |
| 数组下标 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

动态区间

单调队列：5 3 2  
最后答案：5 5

- 我们可以用一个单调递减的队列来解决这个问题，队首元素最大。 数列：1 5 3 2 2 4 1

i=5  
动态区间[3, 5]

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 数列   | 1 | 5 | 3 | 2 | 2 | 4 | 1 |
| 数组下标 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

动态区间

单调队列：3 2  
最后答案：5 5 3

i=6  
动态区间[4, 6]

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 数列   | 1 | 5 | 3 | 2 | 2 | 4 | 1 |
| 数组下标 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

动态区间

单调队列：4  
最后答案：5 5 3 4

i=7  
动态区间[5, 7]

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 数列   | 1 | 5 | 3 | 2 | 2 | 4 | 1 |
| 数组下标 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

动态区间

单调队列：4 1  
最后答案：5 5 3 4 4

i=7  
动态区间[5, 7]

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 数列   | 1 | 5 | 3 | 2 | 2 | 4 | 1 |
| 数组下标 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 动态区间 |   |   |   |   |   |   |   |

单调队列： 4 1  
最后答案： 5 5 3 4 4

```
for (int i = 1; i <= n; i++) {  
    while (队列非空 && 队尾元素 <= a[i]) {  
        弹出队尾元素;  
    }  
    将a[i]加入队尾;  
    如果队首元素已经过气, 将其出队;  
    队首为当前区间答案;  
}
```



给你一个长度为  $n$  的数列  $a$  和一个整数  $c$ ，你可以把它们分成任意长度的若干段，每一段可以去掉前  $\lfloor k/c \rfloor$  小的数字（ $k$  是这一段的长度），求最后剩下的数的和是多少

$$1 \leq n, c \leq 100000$$

- $F[i]$  表示前  $i$  个数划分之后的最小的剩余数字的和
- $f[i] = \min(f[j] + w(j+1, i)) \quad (j \leq i)$
- 其实划分的区间长度要么是1，要么是  $C$
- 所以  $f[i] = \min(f[i-1] + a[i], f[i-c+1] + w(i-c+1, i))$
- $w(i-c+1, i)$  其实就是  $\text{sum}[i] - \text{sum}[i-c] - \text{Min}[i]$
- $\text{Min}[i]$  是第  $i$  个数及其前  $c-1$  个数共  $c$  个数里面的最小值
- 定长连续区间最小值：单调队列维护



1. 优化目标：解决DP转移中的重复计算问题，将时间复杂度从  $O(n^2)$  降至  $O(n)$ 。

2. 适用方程形式：

- 状态转移需满足：

$$dp[i] = \min / \max \{ dp[j] + a[i] + b[j] \} \quad \text{且} \quad L(i) \leq j \leq R(i)$$

- 关键特征：
  - $a[i]$  仅与  $i$  相关,  $b[j]$  仅与  $j$  相关 (分离性)
  - 决策  $j$  的窗口  $[L(i), R(i)]$  随  $i$  单调移动 (如  $j \in [i - k, i]$ )

例3：任务安排

$n$  个任务排成一个序列在一台机器上等待完成（顺序不得改变），这  $n$  个任务被分成若干批，每批包含相邻的若干任务。

从零时刻开始，这些任务被分批加工，第  $i$  个任务单独完成所需的时间为  $t_i$ 。每批任务开始前，机器需要启动时间  $s$ ，完成这批任务所需的时间是各个任务需要时间的总和。

每个任务的费用是它的完成时刻乘以一个费用系数  $f_i$ 。请确定一个分组方案，使得总费用最小。

输入描述

第一行一个正整数  $n$ 。  
第二行是一个整数  $s$ 。  
下面  $n$  行每行有一对数，分别为  $t_i$  和  $f_i$ ，表示第  $i$  个任务单独完成所需的时间是  $t_i$  及其费用系数  $f_i$ 。  
 $1 \leq n \leq 5000, 0 \leq s \leq 50, 1 \leq t_i, f_i \leq 100$ 。

输出描述

一个数，最小的总费用。

样例输入

5  
1  
1 3  
3 2  
4 3  
2 3  
1 4

样例输出

153

方程：

代表前  $i$  个任务分成若干批产生的最小费用  $f[i]$ 。

转移：

$$f_i = \min_{0 \leq j < i} \{f_j + t_i \times (c_i - c_j) + S \times (c_n - c_j)\}$$

$t_i$ ：任务时间前缀和  
 $c_i$ ：费用前缀和

例4：任务安排2

$n$  个任务排成一个序列在一台机器上等待完成（顺序不得改变），这  $n$  个任务被分成若干批，每批包含相邻的若干任务。

从零时刻开始，这些任务被分批加工，第  $i$  个任务单独完成所需的时间为  $t_i$ 。每批任务开始前，机器需要启动时间  $s$ ，完成这批任务所需的时间是各个任务需要时间的总和。

每个任务的费用是它的完成时刻乘以一个费用系数  $f_i$ 。请确定一个分组方案，使得总费用最小。

输入描述

第一行一个正整数  $n$ 。  
第二行是一个整数  $s$ 。  
下面  $n$  行每行有一对数，分别为  $t_i$  和  $f_i$ ，表示第  $i$  个任务单独完成所需的时间是  $t_i$  及其费用系数  $f_i$ 。  
 $1 \leq n \leq 3 \times 10^5, 1 \leq s \leq 2^8, 1 \leq T_i \leq 2^8, 0 \leq C_i \leq 2^8$ 。

输出描述

一个数，最小的总费用。

样例输入

5  
1  
1 3  
3 2  
4 3  
2 3  
1 4

样例输出

153

方程：

$$f_i = \min_{0 \leq j < i} \{f_j + t_i \times (c_i - c_j) + S \times (c_n - c_j)\}$$



$$f_i = \min_{0 \leq j < i} \{f_j - c_j \times (S + t_i)\} + t_i \times c_i + S \times c_n$$

$t_i$ ：任务时间前缀和

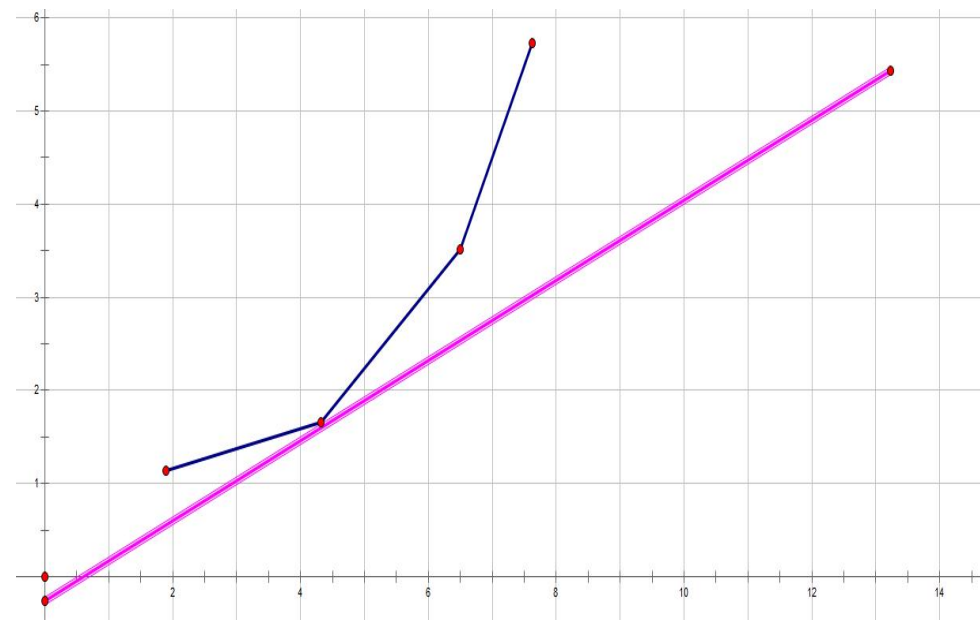
$c_i$ ：费用前缀和

斜率： $k$

截距： $b$

转换一次函数方程形式： $f_j = \overbrace{(t_i + S)}^{\text{斜率: } k} \times \underbrace{c_j}_{\text{自变量: } x} + \overbrace{f_i - c_i \times t_i - S \times c_n}^{\text{截距: } b}$

自变量： $x$



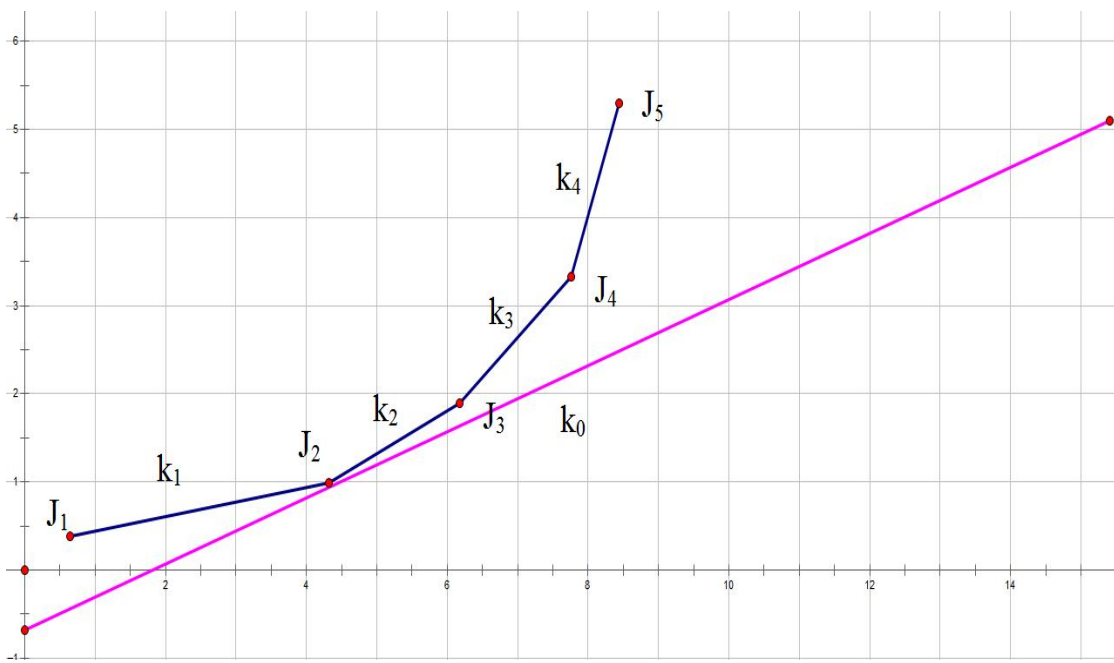
自变量:  $c_j$

因变量:  $f_j$

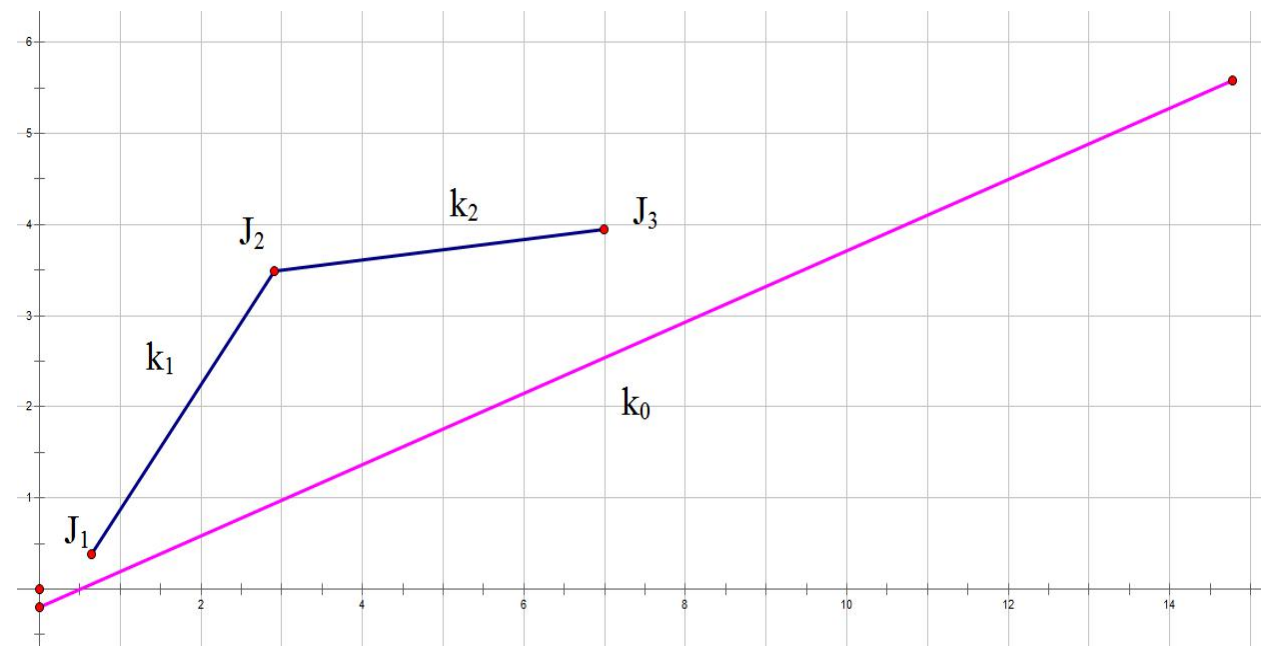
斜率:  $(t_i + S)$

截距:  $f_i - c_i \times t_i - S \times c_n$

例4



当斜率 $k_1 < k_0 < k_2$ 时，此点就是最优转移点



当斜率 $k_1 \geq k_2$ 时， $J_2$ 不会有机会成为最优转移点

- 自变量:  $c_j$
- 因变量:  $f_j$
- 斜率:  $(t_i + S)$
- 截距:  $f_i - c_i \times t_i - S \times c_n$



## 维护下凸壳 (单调队列)

- 凸包性质：决策点需满足 **斜率单调递增**，即对任意三点  $j_1 < j_2 < j_3$ ：

$$\frac{y_{j_2} - y_{j_1}}{x_{j_2} - x_{j_1}} \leq \frac{y_{j_3} - y_{j_2}}{x_{j_3} - x_{j_2}}$$

### 队列操作：

- 队首弹出：若  $j_1$  与  $j_2$  的斜率小于当前  $k = t_i + s$ ，则  $j_1$  不可能是最优解（因  $k$  随  $i$  递增）。
- 队尾维护：插入新点  $i$  时，若尾部三点形成上凸（破坏下凸性），则删除中间点。

```
int hh=0,tt=0;
for(int i=1;i<=n;i++)
{
    while(hh<tt&&f[q[hh+1]]-f[q[hh]]<=(t[i]+s)*(c[q[hh+1]]-c[q[hh]])) hh++; //将所有小于等于斜率 k 的斜率删掉
    int j=q[hh];
    f[i]=f[j]-c[j]*(t[i]+s)+t[i]*c[i]+s*c[n];
    while(hh<tt&&(f[q[tt]]-f[q[tt-1]])*(c[i]-c[q[tt]])>=(f[i]-f[q[tt]])*(c[q[tt]]-c[q[tt-1]])) tt--; //将队尾不符合条件的斜率删掉
    q[++tt]=i;
}

cout<<f[n]<<endl;
```

例5：任务安排3

$n$  个任务排成一个序列在一台机器上等待完成（顺序不得改变），这  $n$  个任务被分成若干批，每批包含相邻的若干任务。

从零时刻开始，这些任务被分批加工，第  $i$  个任务单独完成所需的时间为  $t_i$ 。每批任务开始前，机器需要启动时间  $s$ ，完成这批任务所需的时间是各个任务需要时间的总和。

每个任务的费用是它的完成时刻乘以一个费用系数  $f_i$ 。请确定一个分组方案，使得总费用最小。

输入描述

第一行一个正整数  $n$ 。  
第二行是一个整数  $s$ 。  
下面  $n$  行每行有一对数，分别为  $t_i$  和  $f_i$ ，表示第  $i$  个任务单独完成所需的时间是  $t_i$  及其费用系数  $f_i$ 。  
 $1 \leq n \leq 3 \times 10^5, 1 \leq s \leq 2^8, |T_i| \leq 2^8, 0 \leq C_i \leq 2^8$ 。

输出描述

一个数，最小的总费用。

样例输入

5  
1  
1 3  
3 2  
4 3  
2 3  
1 4

样例输出

153

## 一、核心原理

斜率优化用于优化状态转移方程中存在“既有  $i$  又有  $j$  的乘积项”的 DP 问题（如  $dp[i] = \min\{ dp[j] + a[i]*b[j] \}$ ），通过转化为几何模型（直线截距最小化）和凸包维护，将复杂度从  $O(n^2)$  降至  $O(n)$  或  $O(n \log n)$

## 二、适用场景

### 1. 问题特征：

- 状态转移方程为：

$$dp[i] = \min / \max \{ f(j) + g(i) \cdot h(j) \}$$

其中  $g(i)$  与  $h(j)$  是乘积关系

- 单调性要求（至少满足其一）：
  - 斜率  $k_i$  单调（如  $a[i]$  递增）；
  - 决策点横坐标  $x_j$  单调（如  $b[j]$  递增）

## 1. 斜率 $k_i$ 和 $x_j$ 均单调

- 复杂度:  $O(n)$
- 维护凸包: 单调队列存储决策点, 保证相邻点斜率递增 (下凸壳)
- 操作:
  - 队头弹出: 若斜率  $k_i \geq \text{slope}(q_l, q_{l+1})$ , 则  $q_l$  非最优, 弹出队头
  - 队尾维护: 新点  $i$  入队前, 若  $\text{slope}(q_{r-1}, q_r) \geq \text{slope}(q_r, i)$ , 则弹出  $q_r$  (破坏凸性)

## 2. 斜率 $k_i$ 不单调, $x_j$ 单调

- 复杂度:  $O(n \log n)$
- 维护凸包: 仍用单调队列, 但最优决策点需二分查找。
- 操作: 在凸包上二分找到第一个满足  $\text{slope}(q_m, q_{m+1}) > k_i$  的位置  $m$ , 则  $q_m$  为最优决策点

## 3. $x_j$ 不单调

- 方法 1: CDQ 分治, 离线处理动态凸包
- 方法 2: 平衡树动态维护凸包 (如 Splay 树), 支持插入点并删除非凸点

# 谢 谢 大 家

董国梁

